

AndroMetric: Bridging Multi-Dimensional Software Metrics and Mobile Application Security

Sebastian Siedler
Florida Polytechnic University
Lakeland, FL, USA
ssiedler3117@floridapoly.edu

Karim Elish
Florida Polytechnic University
Lakeland, FL, USA
kelish@floridapoly.edu

Abstract

Android applications are widely used and increasingly targeted by malware authors, posing ongoing challenges for security research and app marketplace vetting. Understanding how internal software characteristics relate to app security is a growing research need, yet large-scale datasets linking code metrics with malicious behavior remain scarce. This paper introduces AndroMetric, a dataset designed to bridge software metrics and mobile application security by capturing multi-dimensional structural characteristics and statically inferred behavioral indicators across benign and malicious Android apps. The dataset includes 31 static code metrics extracted from 39,243 APKs (29,208 malware samples and 10,035 benign apps), along with other metadata. Metrics span six categories: size and dimensional structure, object-oriented design, control-flow complexity, sensitive operation usage, Android Intent patterns, and exception handling. AndroMetric supports diverse research applications including malware detection, code quality comparison, and mining trends in Android malware development.

CCS Concepts

- **Security and privacy** → **Software and application security**;
- **Software and its engineering** → *Automated static analysis*.

Keywords

Software Metrics, Static Code Analysis, Mobile Security, Android Malware, Dataset

ACM Reference Format:

Sebastian Siedler and Karim Elish. 2026. AndroMetric: Bridging Multi-Dimensional Software Metrics and Mobile Application Security. In *23rd International Conference on Mining Software Repositories (MSR '26)*, April 13–14, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3793302.3793329>

1 Introduction

Mobile apps on the Android platform have become ubiquitous, with millions available on Google Play and third-party stores [18]. Unfortunately, this popularity attracts malicious developers who exploit the platform to distribute harmful applications. As a result, understanding how software characteristics relate to the security of mobile applications has become an important research direction in both software engineering and security communities [4–6, 21].

To fill this gap, we introduce **AndroMetric**¹, a dataset that bridges software metrics and mobile application security by providing a large-scale collection of *multi-dimensional software metrics* extracted from 10,035 benign and 29,208 malicious Android applications. AndroMetric includes six categories of metrics: (1) size and dimensional metrics, (2) object-oriented design metrics, (3) complexity metrics, (4) sensitive operation and API usage metrics, (5) Android Intent-launch metrics, and (6) exception-handling metrics. These categories capture both structural and behavioral intent characteristics of applications, enabling new research at the intersection of software quality and security.

The design of AndroMetric is guided by four main objectives. First, *reusability*: the dataset is provided in a standard CSV format, and each record includes the SHA-256 hash of the analyzed APK, enabling researchers to retrieve the original app from public repositories such as AndroZoo [1] for independent verification. Second, *diversity*: AndroMetric includes multiple categories of code metrics—such as size, complexity, object-oriented design, and behavioral intent measures—capturing a wide range of structural characteristics of mobile applications. Third, *security relevance*: the dataset contains metrics extracted from both benign and malicious applications, enabling comparative analyses of software characteristics across different security categories. Finally, *reproducibility*: the dataset construction process and the metrics extraction steps are described to support replication and future extensions by other researchers.

AndroMetric provides software metrics for both benign and malicious Android applications to enable empirical studies that examine how code-level characteristics relate to app security. Researchers can use this dataset to analyze differences in software structure between benign and malware apps, explore potential indicators of malicious design, or benchmark machine learning models for security prediction. We expect AndroMetric to serve as a valuable foundation for future research on software security analytics and metric-based characterization of mobile applications.

In Section 2, we describe the data sources and the process used to construct the dataset. In Section 3, we present the structure of the dataset. Section 4 outlines potential research applications and use cases. Section 5 discusses the challenges and limitations in constructing the dataset. In Section 6, we discuss related datasets and prior work. Finally, Section 7 concludes the paper.

2 Data Collection and Metrics Extraction

To build the AndroMetric dataset, we collected and analyzed 39,243 Android applications, comprising 10,035 benign apps and 29,208



This work is licensed under a Creative Commons Attribution 4.0 International License. *MSR '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2474-9/26/04

<https://doi.org/10.1145/3793302.3793329>

¹<https://zenodo.org/records/17559667>

malicious samples. This section outlines the data sources, preprocessing steps, and the metric extraction process used to construct the dataset.

Benign App Collection. We collected 10,035 benign APKs from two primary sources: the Google Play Store and the AndroZoo repository [1]. AndroZoo provides a large-scale archive of Android applications along with metadata, such as package names, SHA-256 hashes, upload dates, and antivirus scan results. We selected the benign apps from AndroZoo that were not flagged by any antivirus engine to ensure that they represent trustworthy software. Apps were sampled from a variety of categories and sizes to ensure diversity. Each app in the dataset is identified by its SHA-256 hash and package name to allow reproducibility and traceability.

Malware App Collection. The malicious subset of the dataset was constructed using two widely studied malware corpora: the Drebin dataset [2, 17] and the Android Malware Dataset (AMD) [20]. After deduplication based on SHA-256 hashes and package names, we retained approximately 29,208 unique malware APKs. Where available, family labels from the original datasets are preserved to support malware family-specific analysis.

Preprocessing and Decompilation. Each APK was decompiled using ApkTool², a widely used reverse-engineering tool that converts Dalvik Executable (DEX) bytecode into human-readable Smali code. The resulting Smali code was then parsed by the metric engine to extract metrics related to code size, complexity, object-oriented design, sensitive operation usage, and exception handling.

Metrics Extraction. For metrics extraction, we used an open-source static analysis tool developed by Grano et al. [7], originally designed to analyze the relationship between app reviews and internal code quality. This tool, known as the *Android Quality Metrics Extractor*³, computes a broad set of software metrics at the application level by parsing decompiled Smali code. We adapted the tool to address limitations when processing large-scale datasets—such as handling missing metadata and resolving runtime exceptions encountered during batch execution.

Data Organization. The final dataset is released in CSV format. Each row corresponds to one application and includes the following:

- **App identifiers:** SHA-256 hash and package name.
- **Label:** Binary class label (Benign or Malware), with family name for malware samples.
- **Metrics:** A total of 31 code metrics across six categories, including dimensional, complexity, object-oriented, and Android-specific behavioral metrics.

Figure 1 presents the overall workflow of the AndroMetric dataset construction process, outlining the key stages involved in collecting Android apps, extracting metrics, and generating the final structured dataset.

3 Dataset Structure

The AndroMetric dataset contains 39,243 Android apps (10,035 benign and 29,208 malicious), each described by 31 static code metrics along with metadata fields such as SHA-256 hash, package name, source, and class label. All data is provided in flat CSV format for ease of integration with analysis pipelines.

²<https://apktool.org/>

³https://github.com/sealuzh/user_quality/tree/master/code_metrics_scripts

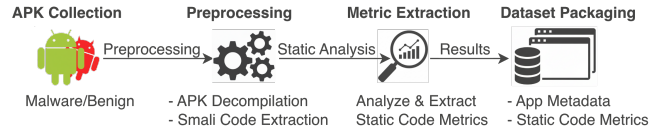


Figure 1: Overview of the AndroMetric dataset construction pipeline, from app collection to dataset generation.

3.1 Code Metrics Description

To support analysis at multiple levels, the metrics are organized into six major categories as follows.

Size and Dimensional Metrics. These metrics reflect the structural volume of an app's codebase. They include total counts of classes, methods, and bytecode instructions, as well as average values such as methods per class and bytecode per method. These values capture the app's implementation scale and density. Benign apps, especially production-grade ones from official stores, tend to exhibit higher values across these dimensions, while malware apps—often smaller or auto-generated—may have more minimal footprints.

Object-Oriented Design Metrics. This group includes object-oriented design indicators, primarily drawn from the Chidamber and Kemerer (CK) metric suite [9]. These include: CBO (Coupling Between Objects), DIT (Depth of Inheritance Tree), NOC (Number of Children), LCOM (Lack of Cohesion of Methods), PPIV (Proportion of Public Instance Variables), and APD (Average Parameter Density). These metrics help assess modularity, inheritance complexity, and coupling. Malware apps may show lower structural sophistication, though repackaged or obfuscated variants may display inflated complexity due to code grafting.

Code Complexity Metrics. We include standard complexity metrics such as McCabe's Cyclomatic Complexity [13] and Weighted Methods per Class (WMC). These reflect control flow complexity and code maintainability. Metrics like cyclomatic and WMC were aggregated at the application level. Prior studies suggest malware often exhibits low maintainability and minimal control flow branching [15]; our dataset enables empirical validation of such claims across a large malware corpus.

Sensitive API and Operation Usage. To capture potentially security-relevant behavior, we include counts of sensitive operations and Android API invocations, such as: `network_op` (network access), `fileio_op` (file I/O), and `sqlite_op` (database usage). These features help profile whether an app accesses private data, performs I/O, or communicates over the network—capabilities often associated with malware. These features complement structural metrics with more semantic indicators of behavior.

Android Intent Launch Metrics. Android apps interact via the Intent system. We include metrics counting how many times apps invoke different "start*" methods (e.g., `start_activity`, `start_service`, `start_intent_for_result`). These values provide insight into how apps initiate inter-component communication. Malware sometimes abuses the Intent system to trigger actions or intercept system broadcasts; this group of features can help surface such patterns.

Table 1: Summary of the static code metrics, organized by their conceptual categories and representative feature columns.

Metric Group	Representative Columns (abbreviated)
Structural / Dimensional	classes, methods, bytecode instructions, methods_per_class, bytecode_per_method
Complexity Metrics	cyclomatic, WMC
Object-Oriented Metrics	NOC, DIT, LCOM, CBO, PPIV, APD
Sensitive Operations / APIs	network_op, sqlite_op, fileio_op
Exception Handling	catch, log, no_action
Intent Launch Metrics	start_activity, start_service, start_intent_for_result, start_activity_result, etc.

Table 2: Dataset schema describing metadata fields and classification labels for each analyzed Android app.

Column Name	Description
sha256	SHA-256 hash of the APK file (unique identifier)
app_name	App/package name
app_source	Origin of the app (e.g., AMD, Drebin, etc.)
family	Malware family name
class	Binary label: Malware or Benign

Exception Handling Patterns. Exception-handling style reflects coding practices and defensive design. We record the number of try-catch blocks, as well as ‘empty catch’ and ‘log’ statements. While benign apps tend to include meaningful exception handling for robustness, malware may suppress errors using blanket or empty catch blocks. These metrics are useful for comparing the quality and intent of control flow error handling across app classes.

Table 1 provides an overview of the six major metric groups captured in AndroMetric, including representative measures for each group. Figure 2 shows representative metric distributions from the six categories, capturing how benign and malicious apps vary across sensitive operations, exception handling patterns, Android-Intent usage, and other code-level characteristics.

3.2 Data Format and Completeness

All metrics are computed consistently across apps to ensure comparability and are stored as integer or floating-point values. No missing values are expected. In addition to the 31 code metrics, each dataset record includes the following metadata:

- sha256: cryptographic hash of the APK
- app_name: application name
- app_source: origin of the sample
- family: malware family name
- class: binary label (Malware or Benign)

The overall structure of the metadata and class labeling is summarized in Table 2, which lists key identifiers such as SHA-256 hashes, package names, source repositories, and class labels distinguishing benign from malicious apps. This format enables researchers to easily filter or segment the dataset and to apply supervised learning techniques or statistical comparisons.

4 Dataset Use Cases

The AndroMetric dataset supports a broad range of research and practical applications. In this section, we highlight a few example use cases enabled by AndroMetric.

Malware Detection and Classification. AndroMetric offers a labeled dataset of static code metrics suitable for training and evaluating machine learning models that distinguish benign apps from malware. Researchers can design and evaluate supervised or unsupervised machine learning models that differentiate malicious from benign apps based on quantitative indicators of structure, complexity, and behavior. The dataset supports a wide variety of modeling paradigms, ranging from traditional classifiers (e.g., Random Forest, SVM) to modern deep learning and graph-based neural architectures. Moreover, the fine-grained metrics enable family-level or variant-level analysis, where structural differences across malware lineages can be examined to enhance explainability and generalization of detection systems.

Code Quality Comparison. The dataset enables comparative studies of coding practices between benign and malicious apps. Prior work has indicated that malicious apps often exhibit lower design quality, reduced modularity, and less rigorous exception handling [15]. With comprehensive metrics, spanning size, complexity, object-oriented design, and component-level attributes, researchers can quantitatively compare code quality between benign and malware apps. Such analyses may reveal recurring design deficiencies in malware and highlight engineering trade-offs in malicious code development.

Risk Assessment and Vetting. App store operators or security analysts can use the dataset to prototype heuristic rules or scoring models. For example, apps with extensive use of sensitive operations and weak error handling may be flagged for deeper review, aiding in early detection or pre-publication screening.

Cross-Dataset Integration and Enrichment. With SHA-256 hashes and package names, AndroMetric can be linked to datasets like AndroZoo [1], VirusTotal [19], or Google Play metadata. Researchers

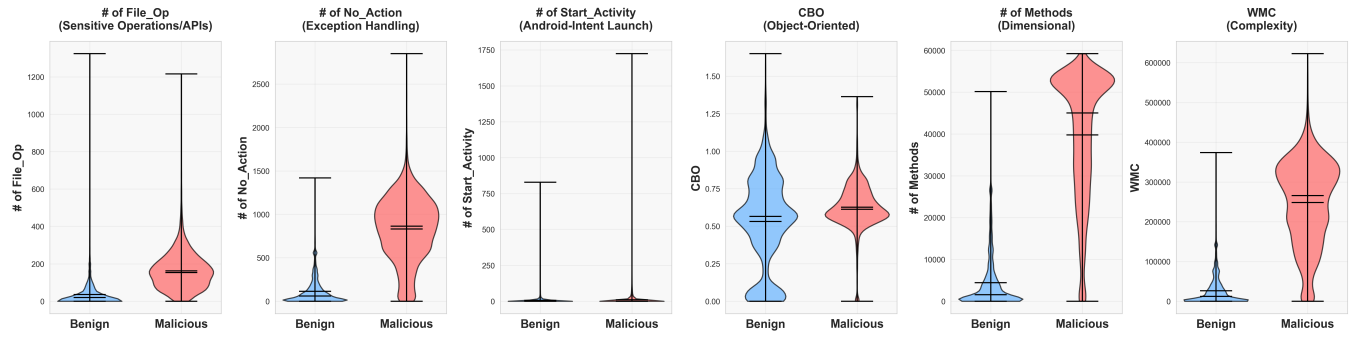


Figure 2: Representative distributions of selected metrics from each category, comparing benign and malicious Android apps.

can augment the dataset with additional layers of contextual information, such as app popularity, update frequency, user ratings, or vulnerability disclosures, facilitating multi-dimensional analyses that combine software engineering metrics with ecosystem-level factors. For instance, studies may investigate how code complexity correlates with app longevity, user trust, or security incidents. Furthermore, the dataset supports repackaging detection research, where metric-based similarity comparisons between original and modified apps can reveal obfuscation or cloning patterns.

5 Limitations

Several factors affected the construction and scope of the AndroMetric dataset. First, some of APKs failed during decompilation or metric extraction due to factors such as corrupted DEX files, or unsupported instructions. These cases were excluded to preserve data quality. Second, static analysis inherently cannot capture dynamic behaviors such as code loaded at runtime, reflective API usage, or native code execution. Consequently, some security-relevant behaviors may not be reflected in the extracted metrics. Third, the malware samples—although diverse—are limited to those available in Drebin and AMD. Lastly, the dataset includes more malware samples than benign ones, which could introduce class imbalance in downstream learning tasks. Appropriate sampling or balancing techniques should be applied when using the dataset for training machine learning models.

6 Related Work and Datasets

Several studies have explored the use of static code metrics to analyze Android apps in the contexts of malware detection and software quality. Early work showed that complexity metrics could help distinguish malware from benign apps [15]. More recent studies have applied Android-oriented structural metrics to classify malware families, revealing that code-level features can capture patterns unique to different malicious variants [3, 6]. Other researchers have examined how these metrics evolve over time to track changes in malware quality [14], or to compare code quality between apps from official and third-party marketplaces [16]. However, none of these studies have considered a comprehensive set of metrics that capture multiple dimensions of the code, nor have they released their datasets, thereby limiting reproducibility and broader analysis.

Various datasets have been introduced to support Android security research. Some, such as the Android Malware Genome [22] and Drebin [2] datasets, primarily provide labeled malware samples. Others, such as CICMalDroid [11], AMD [20], MaMaDroid [12], and Kronodroid [8], provide additional feature vectors derived from static and/or dynamic analyses, including permissions, API calls, manifest attributes, and system calls. While widely used in malware detection tasks, these datasets do not include software metrics that capture internal code structure or complexity. AndroZoo [1] offers a large archive of Android APKs with associated metadata and virus scan results, but does not include static code metrics. Similarly, AndroZooOpen [10] provides open-source Android apps along with package metadata and reviews, but focuses primarily on source code availability rather than detailed internal code metrics. AndroMetric complements these resources by providing a publicly available, large-scale labeled dataset of multi-dimensional code metrics for both benign and malicious apps, aiming to bridge structural quality indicators with security classification.

7 Conclusion and Future Work

This paper presented AndroMetric, a publicly available dataset that bridges software metrics and mobile application security. It provides multi-dimensional static code metrics for 39,243 Android applications, covering both benign and malicious samples. The dataset captures six categories of structural and inferred behavioral characteristics, enabling empirical studies in areas such as malware detection, code quality comparison, and security risk assessment. The dataset also includes SHA-256 hashes and package names for each app to support cross-referencing with other datasets and ensure reproducible experimentation.

In future work, we plan to expand the dataset with newer malware samples and incorporate additional metrics derived from dynamic or hybrid analysis. We also aim to provide additional tools and visualization support to help researchers more easily analyze and extend the dataset.

References

- [1] Kevin Allix, Tegawendé F Bissyandé, Jacques Klein, and Yves Le Traon. 2016. AndroZoo: Collecting millions of Android apps for the research community. In *Proceedings of the 13th International Conference on Mining Software Repositories (MSR)*. ACM, 468–471.
- [2] Daniel Arp, Michael Spreitzenbarth, Malte Hübner, Hugo Gascon, and Konrad Rieck. 2014. Drebin: Efficient and Explainable Detection of Android Malware in

- Your Pocket. In *Proceedings of the 21st Annual Network and Distributed System Security Symposium (NDSS)*. San Diego, CA, USA.
- [3] William Blanc, Lina Hashem, Karim Elish, and Hussain Almohri. 2019. Identifying Android Malware Families Using Android-Oriented Metrics. In *Proceedings of the IEEE International Conference on Big Data (Big Data)*. 4708–4713.
 - [4] Haipeng Cai and Barbara Ryder. 2017. Understanding Android Application Programming and Security: A Dynamic Study. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 364–375.
 - [5] Fahimeh Ebrahimi, Miroslav Tushev, and Anas Mahmoud. 2021. Mobile app privacy in software engineering research: A systematic mapping study. *Information and Software Technology* 133 (2021), 106466.
 - [6] Karim Elish, Mahmoud Elish, and Hussain Almohri. 2022. Lightweight, Effective Detection and Characterization of Mobile Malware Families. *IEEE Trans. Comput.* 71, 11 (2022), 2982–2995.
 - [7] Giovanni Grano, Andrea Di Sorbo, Francesco Mercaldo, Corrado A. Visaggio, Gerardo Canfora, and Sebastiano Panichella. 2017. Android Apps and User Feedback: A Dataset for Software Evolution and Quality Improvement. In *Proceedings of the 2nd ACM SIGSOFT International Workshop on App Market Analytics*. 8–11.
 - [8] Alejandro Guerra-Manzanares, Hayretin Bahsi, and Sven Nömm. 2021. KronoDroid: Time-based Hybrid-featured Dataset for Effective Android Malware Detection and Characterization. *Computers & Security* 110 (2021), 102399. <https://www.sciencedirect.com/science/article/pii/S0167404821002236>
 - [9] C. Kemerer and S. Chidamber. 1994. A Metrics Suite for Object-Oriented Design. *IEEE Transactions on Software Engineering* 20, 06 (1994).
 - [10] Pei Liu, Li Li, Yanjie Zhao, Xiaoyu Sun, and John Grundy. 2020. AndroZooOpen: Collecting Large-scale Open Source Android Apps for the Research Community. In *Proceedings of the 17th IEEE/ACM International Conference on Mining Software Repositories (MSR)*. 548–552.
 - [11] Samaneh MahdaviFar, Andi Fitriah Abdul Kadir, Rasool Fatemi, Dima Alhadidi, and Ali A. Ghorbani. 2020. Dynamic Android Malware Category Classification using Semi-Supervised Deep Learning. In *Proceedings of the IEEE Intl. Conf. on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCom/CyberSciTech)*. 515–522.
 - [12] Enrico Mariconti, Lucky Onwuzurike, Panagiotis Andriotis, Emiliano De Cristofaro, Gordon Ross, and Gianluca Stringhini. 2017. MAMADROID: Detecting Android Malware by Building Markov Chains of Behavioral Models. In *Proceedings of the Annual Symposium on Network and Distributed System Security (NDSS)*.
 - [13] T. McCabe. 1976. A Complexity Measure. *IEEE Transactions on Software Engineering* 2, 04 (1976).
 - [14] Francesco Mercaldo, Andrea Di Sorbo, Corrado Aaron Visaggio, Aniello Cimitile, Fabio Martinelli, and Canfora Gerardo. 2018. An exploratory study on the evolution of Android malware quality. *Journal of Software: Evolution and Process* 30, 11 (Nov. 2018), 1 pages.
 - [15] Mykyta Protsenko and Henning Müller. 2014. Android malware detection based on software complexity metrics. In *Proceedings of the International Conference on Trust, Privacy and Security in Digital Business*. Springer, 212–219.
 - [16] Mateus Rebouças, Gustavo Pinto, and Uirá Kulesza. 2019. Investigating mobile applications quality in official and third-party marketplaces. In *Proceedings of the 14th International Conference on Software Technologies (ICSOFT)*. 92–103.
 - [17] Michael Spreitzenbarth, Florian Ehtler, Thomas Schreck, Felix C. Freiling, and Johannes Hoffmann. 2013. MobileSandbox: Looking Deeper into Android Applications. In *Proceedings of the 28th Annual ACM Symposium on Applied Computing (SAC)*. ACM, Coimbra, Portugal.
 - [18] Statista Research Department. 2024. Number of available applications in the Google Play Store from December 2009 to September 2024. <https://www.statista.com/statistics/266210/number-of-available-applications-in-the-google-play-store/>.
 - [19] VirusTotal. 2025. *VirusTotal: Online Virus and Malware Scanner*. <https://www.virustotal.com/>
 - [20] Fengguo Wei, Yiping Li, Sankardas Roy, Xinming Ou, and Wu Zhou. 2017. Deep Ground Truth Analysis of Current Android Malware. In *Proceedings of the 14th International Conference on the Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*.
 - [21] Meng Xu, Chengyu Song, Yang Ji, Ming-Wei Shih, Kangjie Lu, Cong Zheng, Ruian Duan, Yeongjin Jang, Byoungyoung Lee, Chenxiong Qian, Sangho Lee, and Taesoo Kim. 2016. Toward Engineering a Secure Android Ecosystem: A Survey of Existing Techniques. *ACM Computing Surveys* 49, 2 (2016).
 - [22] Yajin Zhou and Xuxian Jiang. 2012. Dissecting Android Malware: Characterization and Evolution. In *Proceedings of the IEEE Symposium on Security and Privacy*. 95–109.